Homework 5

Alex Ojemann

1. I'm measuring what method works best by taking the product of its error and time taken. The lower the score, the better it works because we want the best combination of low cost and high accuracy. Overall, LU had the lowest scores, as its error was too small for the computer to observe in each matrix except for 3. Relatively speaking, Jacobi was the best for matrix 2 as its score was 0 along with LU, LU was best for matrix 3 as its score was very small while the others were extremely large, and Gauss-Seidel was best for matrix 4 because its score was lower than Jacobi's which it wasn't for any of the other matrices. Matrix 5 is nonsingular, so these methods couldn't be applied to it.

2. Fixed point methods converge quicker when the initial guess is closer to the true value. The Gauss-Seidel method used an initial guess. At first I tried 0 and it converged very slowly but I kept trying different numbers and eventually 0.27 resulted in a much faster convergence.

3. I am matrix 3. I am not a diagonally dominant matrix, so Jacobi and Gauss Seidel will be very inefficient for me. Of the three methods implemented, LU is by far the best. This is reflected in the value of time multiplied by error, as this value is far, far smaller for LU than Jacobi or Gauss-Seidel.

```
import numpy as np
import math
import random
with open('mat1-1.txt', 'r') as f:
    m1 = [[float(num) for num in line.split(',')] for line in f]
with open('mat2-1.txt', 'r') as f:
    m2 = [[float(num) for num in line.split(',')] for line in f]
with open('mat3-1.txt', 'r') as f:
    m3 = [[float(num) for num in line.split(',')] for line in f]
with open('mat4-1.txt', 'r') as f:
    m4 = [[float(num) for num in line.split(',')] for line in f]
with open('mat5-1.txt', 'r') as f:
    m5 = [[float(num) for num in line.split(',')] for line in f]
def randomList(numElements):
    myList = []
    for i in range(0,numElements):
        myList.append(random.uniform(0,1))
    return myList
b1=randomList(len(m1))
b2=randomList(len(m2))
b3=randomList(len(m3))
b4=randomList(len(m4))
b5=randomList(len(m5))
s1=np.linalg.solve(m1,b1)
s2=np.linalg.solve(m2,b2)
s3=np.linalg.solve(m3,b3)
s4=np.linalg.solve(m4,b4)
#s5=np.linalg.solve(m5,b5)
#s5 is a singular matrix so running np.linalg.solve on it results in an error
```

```
def LU(a, b):
    n=len(b)
    lower=[[0 for x in range(0,n)] for y in range(n)]
    upper=[[0 for x in range(0,n)] for y in range(n)]
    for i in range(0,n):
        for j in range(i,n):
            sum=0
            for k in range(0,i):
                sum+=(lower[i][k]*upper[k][j])
            upper[i][j]=a[i][j]-sum
        for j in range(i,n):
            if (i==j):
                lower[i][i]=1
            else:
                sum=0
                for k in range(0,i):
                    sum+=(lower[j][k]*upper[k][i])
                lower[j][i]=(a[j][i]-sum)/(upper[i][i])
    x=[0 for x in range(0,n)]
    y=[0 for x in range(0,n)]
    #forward sub on lower triangular matrix to find y
    for i in range(0,n):
        y[i]=(b[i]-(np.dot(y,lower[i])))/(lower[i][i])
        #print("%f"%(y[i]))
    #backward sub on upper triangular matrix to find x
    for i in range(n,0,-1):
        x[i-1]=(y[i-1]-(np.dot(x,upper[i-1])))/(upper[i-1][i-1])
    return x
```

```
def jacobi(a,b):
    n=25
    x=[0 for x in range(0,len(a))]
    d=np.diag(a)
    r=a-np.diagflat(d)
    for i in range(0,n):
        x=(b-np.dot(r,x))/d
    return x
def gauss_seidel(a,b):
    n=len(a)
    x=[0.27 for x in range(0,n)]
    for i in range(0,n):
        current=b[i]
        for j in range(0,n):
            if(i!=j):
                current-=a[i][j]*x[j]
        x[i]=current/a[i][i]
    return x
```

```
import time
mark1=time.perf_counter()
LUs1=LU(m1,b1)
mark2=time.perf_counter()
jacobis1=jacobi(m1,b1)
mark3=time.perf_counter()
gauss_seidels1=gauss_seidel(m1,b1)
mark4=time.perf_counter()
LUTime1=mark2-mark1
jacobiTime1=mark3-mark2
gaussTime1=mark4-mark3
x=[]
LUError1=0
jacobiError1=0
gaussError1=0
for i in range(0,len(s1)):
    LUs1[i]-=s1[i]
    if(LUs1[i]<0):
        LUs1[i]*=-1
    jacobis1[i]-=s1[i]
    if(jacobis1[i]<0):
        jacobis1[i]*=-1
    gauss_seidels1[i]-=s1[i]
    if(gauss_seidels1[i]<0):
        gauss_seidels1[i]*=-1
    x.append(i)
for i in range(0,len(s1)):
    LUError1+=LUs1[i]
    jacobiError1+=jacobis1[i]
    gaussError1+=gauss_seidels1[i]
print("%f %f %f"%(LUError1*LUTime1,jacobiError1*jacobiTime1,gaussError1*gaussTime1))
```

```
0.000000 0.206774 2.244406
```

```
import time
mark1=time.perf_counter()
LUs2=LU(m2,b2)
mark2=time.perf_counter()
jacobis2=jacobi(m2,b2)
mark3=time.perf_counter()
gauss_seidels2=gauss_seidel(m2,b2)
mark4=time.perf_counter()
LUTime2=mark2-mark1
jacobiTime2=mark3-mark2
gaussTime2=mark4-mark3
x=[]
LUError2=0
jacobiError2=0
gaussError2=0
for i in range(0,len(s2)):
    LUs2[i]-=s2[i]
    if(LUs2[i]<0):
        LUs2[i]*=-1
    jacobis2[i]-=s2[i]
    if(jacobis2[i]<0):
        jacobis2[i]*=-1
    gauss_seidels2[i]-=s2[i]
    if(gauss_seidels2[i]<0):
        gauss_seidels2[i]*=-1
    x.append(i)
for i in range(0,len(s2)):
    LUError2+=LUs2[i]
    jacobiError2+=jacobis2[i]
    gaussError2+=gauss_seidels2[i]
print("%f %f %f"%(LUError2*LUTime2,jacobiError2*jacobiTime2,gaussError2*gaussTime2))
```

```
0.000000 0.000000 1.840961
```

```
import time
mark1=time.perf_counter()
LUs3=LU(m3,b3)
mark2=time.perf_counter()
jacobis3=jacobi(m3,b3)
mark3=time.perf_counter()
gauss_seidels3=gauss_seidel(m3,b3)
mark4=time.perf_counter()
LUTime3=mark2-mark1
jacobiTime3=mark3-mark2
gaussTime3=mark4-mark3
x=[]
LUError3=0
jacobiError3=0
gaussError3=0
for i in range(0,len(s3)):
    LUs3[i]-=s3[i]
    if(LUs3[i]<0):
        LUs3[i]*=-1
    jacobis3[i]-=s3[i]
    if(jacobis3[i]<0):
        jacobis3[i]*=-1
    gauss_seidels3[i]-=s3[i]
    if(gauss_seidels3[i]<0):
        gauss_seidels3[i]*=-1
    x.append(i)
for i in range(0,len(s3)):
    LUError3+=LUs3[i]
    jacobiError3+=jacobis3[i]
    gaussError3+=gauss_seidels3[i]
print("%f %f %f"%(LUError3*LUTime3,jacobiError3*jacobiTime3,gaussError3*gaussTime3))
```

```
0.018047 749347085726614612200244977384654475523597537232867642221946033294690914867942679211831965746412305182300335306154335886121007802114636049467110403303580631040.000000 inf
```

```
import time
mark1=time.perf_counter()
LUs4=LU(m4,b4)
mark2=time.perf_counter()
jacobis4=jacobi(m4,b4)
mark3=time.perf_counter()
gauss_seidels4=gauss_seidel(m4,b4)
mark4=time.perf_counter()
LUTime4=mark2-mark1
jacobiTime4=mark3-mark2
gaussTime4=mark4-mark3
x=[]
LUError4=0
jacobiError4=0
gaussError4=0
for i in range(0,len(s4)):
    LUs4[i]-=s4[i]
    if(LUs4[i]<0):
        LUs4[i]*=-1
    jacobis4[i]-=s4[i]
    if(jacobis4[i]<0):
        jacobis4[i]*=-1
    gauss_seidels4[i]-=s4[i]
    if(gauss_seidels4[i]<0):
        gauss_seidels4[i]*=-1
    x.append(i)
for i in range(0,len(s4)):
    LUError4+=LUs4[i]
    jacobiError4+=jacobis4[i]
    gaussError4+=gauss_seidels4[i]
print("%f %f %f"%(LUError4*LUTime4,jacobiError4*jacobiTime4,gaussError4*gaussTime4))
```

```
0.000000 0.124456 0.002571
```